```
    client.BaseAddress = new Uri("https://example.com/rates/");   ❷

       var response = await client.GetAsync("latest");            ❸

    response.EnsureSuccessStatusCode();                           ❹
    return await response.Content.ReadAsStringAsync();            ❺
});
app.Run();
```

❶ Wrapping the HttpClient in a using declaration means it is disposed at the end of the scope.

❷ Configures the base URL used to make requests using the HttpClient

❸ Makes a GET request to the exchange rates API

❹ Throws an exception if the request was not successful

❺ Reads the result as a string and returns it from the action method

`HttpClient` is special, and you shouldn't use it like this! The problem is due primarily to the way the underlying protocol implementation works. Whenever your computer needs to send a request to an HTTP server, you must create a connection between your computer and the server. To create a connection, your computer opens a port, which has a random number between 0 and 65,535, and connects to the HTTP server's IP address and port, as shown in figure 33.1. Your computer can then send HTTP requests to the server.

DEFINITION The combination of IP address and port is called a *socket*.

The main problem with the `using` statement/declaration and `HttpClient` is that it can lead to a problem called *socket exhaustion*, illustrated in figure 33.2. This happens when all the ports on your computer have been used up making other HTTP connections, so your computer can't make any more requests. At that point, your application will hang, waiting for a socket to become free—a bad experience!

